



Documentación Técnica con IA — Fundamentos y Tipos

Ejercicio de Clase · Ejercicio 1 de 3 · Bloques 3.1 — 3.2 — 3.3

Nombre del alumno/a	Angel Echenique
Fecha	03/04/2026
🔗 Enlace GitHub	

📋 Instrucciones: Responde con lo que vimos en clase. Puedes consultar tus apuntes. En el Bloque 5 deberás investigar por tu cuenta y citar la fuente. ¡Solo demostrar que entendiste, no hay que escribir código!

BLOQUE 1 — ¿Por qué documentar con IA? (3.1)

En clase vimos que la documentación es una de las tareas más olvidadas y a la vez más críticas. Completa la tabla con las razones por las que la IA transforma este proceso:

Motivo / Beneficio	¿Qué hace exactamente la IA?	¿Qué problema resuelve?
Ahorra tiempo	Genera documentación automáticamente: docstrings, comentarios, ejemplos.	Evita que el programador pierda tiempo escribiendo documentación manual.
Mejora la claridad del código	Explica lógica compleja en lenguaje simple y estructurado.	Hace que el código sea más fácil de entender.
Facilita el mantenimiento	Describe funciones, flujos y dependencias.	Reduce errores al modificar el código.
Ayuda a entender código heredado	Analiza código antiguo o sin comentarios y lo explica.	Permite entender proyectos que no hiciste tú.
Genera documentación consistente	Mantiene un mismo estilo y estructura.	Evita documentación desordenada o incoherente.



Enseña buenas prácticas técnicas	Muestra cómo documentar correctamente (parámetros, retornos, etc.).	Ayuda a aprender a documentar como profesional.
Acelera el onboarding de nuevos miembros	Resume el proyecto y explica su estructura.	Reduce el tiempo de aprendizaje de nuevos desarrolladores.

1.2 — La IA como 'traductor personal' del código

En clase se describió a la IA como un 'traductor personal'. Explica con tus propias palabras qué significa esta analogía y pon un ejemplo concreto:

Concepto	Tu explicación + ejemplo
¿Qué significa que la IA es un 'traductor'?	Toma el código, que puede ser difícil de entender, y lo convierte en explicaciones claras para facilitar la comprensión.
Ejemplo concreto de uso	Tengo una función sin comentarios y no sé qué hace. Le paso el código a la IA y me explica paso a paso qué hace cada parte.

BLOQUE 2 — Tipos de documentación que puede generar la IA (3.2)

Vimos que la IA puede generar diferentes tipos de documentación. Clasifica y describe cada uno:

Tipo	¿Qué contiene?	¿Dentro/fuera del código?	Ejemplo de uso
Comentarios en línea	Explican líneas o bloques específicos del código.	Dentro del código.	# Validar que el usuario exista en la base de datos.
Docstrings	Descripción formal de funciones/clases: parámetros, retorno, errores, ejemplos.	Dentro del código.	Función Python con docstring estilo Google.
README	Descripción del proyecto, instalación, uso, dependencias.	Fuera del código.	Archivo README.md en GitHub.
Documentación de APIs	Endpoints, parámetros, respuestas, errores.	Fuera del código.	Documentación de una API REST.
Manuales técnicos	Arquitectura, diseño, decisiones técnicas.	Fuera del código.	Documento explicando cómo funciona el sistema.
Guías de uso	Pasos para usar la aplicación, ejemplos, errores comunes.	Fuera del código.	Tutorial de cómo usar una app.
OpenAPI / Swagger	Especificación estructurada de APIs (endpoints, schemas, responses).	Fuera del código (aunque generado desde código).	Archivo YAML/JSON para Swagger.

JSDoc / TypeDoc	Documentación automática para JavaScript/TypeScript.	Dentro del código (comentarios especiales).	<code>/** @param {string} name */</code>
Sphinx (Python)	Documentación estructurada a partir de docstrings.	Fuera del código (generada).	Web de documentación de un proyecto Python.

2.2 — Documentación orientada a roles

La IA adapta la documentación según quién la lee. Indica qué enfoque daría la IA para cada rol:

Rol del lector	¿Qué información prioriza la IA?	Ejemplo de contenido específico
Desarrollador Backend	Lógica de negocio, seguridad, base de datos.	Validación de usuarios, autenticación, queries.
Desarrollador Frontend	Respuestas JSON, endpoints, errores.	Cómo consumir una API, estructura de datos.
QA / Tester	Casos límite, errores, validaciones.	Inputs inválidos, errores esperados.
DevOps	Configuración, variables de entorno, despliegue.	Puertos, dependencias, scripts de deploy.

BLOQUE 3 — Cómo pedir documentación a la IA (3.3)

Un buen prompt para pedir documentación debe incluir elementos clave. Completa la tabla:

Elemento del Prompt	¿Por qué es importante?	Ejemplo de texto
Rol (Role)	Define cómo debe actuar la IA.	Actúa como un experto en documentación técnica.
Contexto (Context)	Da información del proyecto o situación.	Estamos trabajando en una aplicación de e-commerce.



Acción (Action)	Indica exactamente qué debe hacer la IA.	Genera un docstring para esta función.
Formato (Format)	Define cómo debe ser la salida.	En formato docstring estilo Google.
Restricciones (Constraints)	Limita o especifica condiciones.	Incluye ejemplos y no uses lenguaje complejo.
Audiencia (Audience)	Define para quién es la documentación.	Dirigido a estudiantes de programación.

3.2 — Construyendo el prompt perfecto

Tenemos esta función Python sin documentar. Escribe el prompt completo que usarías para pedir a la IA un docstring profesional:

```
def calcular_descuento(precio, porcentaje, cliente_premium=False):  
    if cliente_premium:  
        porcentaje += 10  
    descuento = precio * (porcentaje / 100)  
    return round(precio - descuento, 2)
```

Escribe aquí tu prompt completo para esta función:

Actúa como un experto en documentación técnica en Python.

Estamos trabajando en un sistema de ventas donde se aplican descuentos a productos según el tipo de cliente y quiero que me generes un docstring profesional estilo google para la siguiente función:

```
def calcular_descuento(precio, porcentaje, cliente_premium=False):  
    if cliente_premium:  
        porcentaje += 10  
    descuento = precio * (porcentaje / 100)  
    return round(precio - descuento, 2)
```

Que quiero que contenga esta documentación:

1. Nivel de detalle: medio (para estudiantes de programación)
2. Incluir:
 - Descripción clara del propósito

- Parámetros con tipo y explicación
 - Valor de retorno
 - Posibles errores
 - Ejemplos de uso
3. Explica de forma clara y didáctica
 4. Incluye también notas sobre posibles validaciones y casos límite.

BLOQUE 4 — Documentación integrada en el Pipeline CI/CD

Vimos cómo la documentación generada por IA puede integrarse en el pipeline de CI/CD. Relaciona cada etapa del pipeline con la acción documental que realiza la IA:

Etapa del Pipeline	¿Qué documentación genera la IA?	¿Por qué es útil en esta etapa?
Al crear un Pull Request	Resumen de cambios, descripción del código modificado.	Ayuda a los revisores a entender rápidamente qué se hizo.
Al detectar baja cobertura de tests	Explicación de funciones sin testear y sugerencias.	Permite mejorar la calidad del código.
Antes de un despliegue	Documentación de funcionalidades nuevas o cambios.	Asegura que todo esté claro antes de pasar a producción.
Tras un fallo en el pipeline	Explicación del error y posibles causas.	Facilita la depuración (debugging).
Al finalizar el pipeline con éxito	Resumen final del build/deploy.	Sirve como registro y documentación del proceso.

4.2 — Verdadero o Falso

Marca V (Verdadero) o F (Falso) y justifica brevemente las que sean Falsas:

V/F	Afirmación	Justificación (solo si es Falsa)
-----	------------	----------------------------------

F	La IA puede generar docstrings sin necesidad de ver el código completo.	La IA necesita ver el código para entender qué hace la función y generar una documentación correcta.
V	La documentación generada por IA siempre debe ser revisada antes de publicarse.	
V/F	Afirmación	Justificación (solo si es Falsa)
F	Los docstrings de estilo Google, NumPy y JSDoc son exactamente iguales.	Cada estilo tiene una estructura diferente y se usa en distintos lenguajes o contextos.
V	La IA puede adaptar el nivel de detalle según el público objetivo.	
F	Un README generado por IA no necesita ninguna modificación para ser profesional.	La IA puede cometer errores o generalizar, por lo que siempre debe revisarse y adaptarse.
V	La IA puede generar documentación OpenAPI compatible con Swagger UI.	

BLOQUE 5 — Investigación propia (cita la fuente)

□ Investiga: Este bloque requiere que investigues por tu cuenta. Usa una fuente fiable y cita siempre de dónde lo sacaste (URL, nombre del artículo o documentación oficial).

5.1 — Herramienta de documentación automática

Investiga una herramienta real de documentación automática con IA (Sphinx, MkDocs, Mintlify, Doxygen...). Describe sus funcionalidades y cómo se integra en un pipeline:

Campo	Tu respuesta
Nombre de la herramienta	Sphinx
¿Integra IA?	No directamente, pero puede integrarse con herramientas de IA para generar docstrings automáticamente.

Lenguajes compatibles	Principalmente Python
Compatible con pipelines CI/CD	Sí, se puede integrar con GitHub Actions, GitLab CI, etc.
Fuente consultada (URL)	https://www.sphinx-doc.org
Fecha de consulta	03/04/2026

5.2 — Caso real: empresa que usa IA para documentar

Busca un caso real de empresa o proyecto open source que use IA para generar documentación técnica:

Campo	Tu respuesta
Empresa o proyecto	GitHub Copilot
Fuente (URL, artículo, blog...)	https://github.blog/2023-03-22-github-copilot-x-the-ai-powered-developer-experience/
Fecha de consulta	03/04/2026

Análisis del bloque 5

La herramienta **Sphinx** permite generar documentación técnica estructurada a partir de **docstrings**, lo que facilita la creación de documentación clara, organizada y profesional en proyectos desarrollados en **Python**. Además, su integración con **pipelines** de CI/CD permite automatizar la generación y actualización de la documentación.

Por otro lado, **GitHub Copilot** demuestra cómo la **inteligencia artificial** puede integrarse directamente en el flujo de desarrollo, ayudando a generar comentarios, docstrings y explicaciones de código de manera automática, mejorando la productividad del desarrollador.



Conclusion del bloque 5

La inteligencia artificial acelera significativamente el proceso de documentación y mejora la comprensión del código, especialmente en proyectos complejos o heredados. Sin embargo, la documentación generada debe ser siempre revisada por el desarrollador para garantizar su precisión, coherencia y adecuación al contexto del proyecto.

Módulo 3 · Documentación Técnica con IA · Dicampus